```
  <tbody>
    <% @project.tasks.each do |task| %>
      <tr>
        <td class="name"><%= task.title %></td>
        <td class="size"><%= task.size %></td>
        <td class="completed"><%= task.completed_at %></td>
      </tr>
    <% end %>
  </tbody>
</table>

<h3>New Task</h3>

<%= form_for Task.new(project_id: @project.id) do |f| %>
  <%= f.hidden_field :project_id %>
  <%= f.label :title, "Task" %>
  <%= f.text_field :title %>
  <%= f.label :size %>
  <%= f.select :size, [1, 2, 3, 4, 5] %>
  <%= f.submit "Add Task" %>
<% end %>
```

At this point you fail because the create task doesn't exist on TasksController. That means you need to create new logic.

This feels like it'll be boilerplate enough not to need additional tests. Often, at this point I'll do a mini spike of the controller method to see how much complexity is called for. In this case the controller method is really simple:

integration/01/app/controllers/tasks_controller.rb

```
class TasksController < ApplicationController
  def create
    @task = Task.new(params[:task].permit(:project_id, :title, :size))
    redirect_to(@task.project)
  end
end
```

You don't have any obvious error cases here—Task has no validations. And even if you did have error cases, the remedy would be to go back to the project page anyway. So I'm willing to leave this for now. (In practice, I'd add an error message in the failure case.)

The next error is Capybara::ElementNotFound: Unable to find css "#task_3".

In the original Capybara test you're identifying each row by the order of the task, so the three rows will have DOM IDs task_1, task_2, and task_3. Not only don't you have those IDs in the template, but you don't even have a mechanism for ordering the tasks.